

Assignment 4: Agentic AI Application

AKINOLA, Lanre Olusegun

Student ID: 12349031

Course: 700.390 Advanced Topics in Neurocomputing:
Explainability, Neuro-Symbolic Computing, AI Agents, LLM,
Quantum Computing, PINN

1 Introduction

This assignment builds a simple agentic AI application in Python. The goal is to create a system that reads a user question, decides which tool to use, calls that tool, checks the result, and produces a clear final answer. The agent works across different types of questions without being told in advance which tool to use.

Agentic AI refers to systems that can take actions on their own to complete a goal [1]. In this case, the agent is kept simple and readable. It uses keyword-based tool selection, three separate tool functions, and a clear trace that shows every step. The focus is on understanding how an agent loop works rather than on building something with a large language model.

2 System Architecture

The system has two main parts: the tools and the agent logic.

2.1 Tools

Three tools are implemented as separate Python functions:

- `search_web(query)`: Searches the internet using the DuckDuckGo API [2]. This is adapted from the `search_api.py` script provided in the course materials. It returns up to five results with title, snippet, and URL.
- `convert_currency(amount, from, to)`: Converts a currency amount using the Frankfurter API [3]. It calls `https://api.frankfurter.app/latest` with `from` and `to` parameters and returns the rate, converted amount, and date.
- `get_multiple_rates(base, targets)`: Same API but fetches several currencies in one call. Used for comparison questions.
- `calculate(expression)`: Evaluates a simple math expression using Python's `eval()` with a restricted namespace (only `math` module functions allowed, no builtins).

2.2 Agent Logic

The agent uses a function called `select_tool(question)` that reads the question and matches keywords to decide which tool to call. The decision rules are:

- If the question contains currency codes (EUR, USD, GBP) **and** phrases like “how much is” or “equals”, it uses the `currency_calculate` path (two steps: get rate, then compute).

- If it contains at least two different currency codes **and** “compare”, it uses `multi_currency`.
- If it has a currency code **and** “convert” or “exchange”, it uses `currency`.
- Everything else goes to `search_web`.

The `run_agent(question)` function then calls the selected tool, observes the result, and formats a final answer. It also prints a full trace showing every step.

2.3 Agent Trace Format

For every question the agent prints:

```
QUESTION : <the user's question>
TOOL      : <selected tool name>
OBSERVE   : <what the tool returned>
ANSWER    : <final human-readable answer>
TIME      : <seconds taken>
```

3 Results

The agent was tested on all seven example questions from the assignment. Table 1 shows the tool selected and the time taken for each question.

Table 1: Agent performance on all 7 example questions

#	Question	Tool	Time
1	What is the capital of Austria, and what is its current population?	search	1.76s
2	Convert 100 EUR to USD.	currency	0.60s
3	Search for recent information about Agentic AI (3 sentences)	search	1.62s
4	If 1 EUR equals X USD, how much is 250 EUR in USD?	currency.calculate	0.55s
5	Find information about MCP and why it is useful for agent tools	search	2.01s
6	Compare EUR to GBP and USD using live exchange-rate data	multi_currency	0.57s
7	What is Tesla's current stock price?	search	1.47s

The following subsections show the full agent trace for each question.

3.1 Question 1 – Capital of Austria

```
QUESTION : What is the capital of Austria, and what is its current population?
TOOL      : search
OBSERVE   : 5 result(s) returned from DuckDuckGo
ANSWER    : It is Austria's primate city, with just over two million inhabitants
            .
            Its larger metropolitan area has a population of nearly 2.9 million,
            representing nearly one third of Austria's population. The country
            occupies an area of 83,879 km2 and has a population of about 9.2
            million.
TIME      : 1.76s
```

3.2 Question 2 – Convert 100 EUR to USD

```
QUESTION : Convert 100 EUR to USD.
TOOL      : currency
OBSERVE   : rate=1.1383, converted=113.83, date=2026-07-01
ANSWER    : 100.0 EUR = 113.83 USD (1 EUR = 1.1383 USD, as of 2026-07-01)
TIME      : 0.60s
```

3.3 Question 3 – Agentic AI Information

```
QUESTION : Search for recent information about Agentic AI and summarize it
           in three sentences.
TOOL      : search
OBSERVE   : 5 result(s) returned from DuckDuckGo
ANSWER    : Today, attention has shifted to the next evolution of generative AI:
           AI agents or agentic AI, a new breed of AI systems that are semi- or
           fully autonomous and thus able to plan, make decisions, and execute
           tasks with minimal human intervention. (Source: Forbes, 2026)
TIME      : 1.62s
```

3.4 Question 4 – Multi-step: 250 EUR to USD

This question required two tool calls: first to get the EUR/USD rate, then to multiply.

```
QUESTION : If 1 EUR equals X USD, how much is 250 EUR in USD?
TOOL      : currency_calculate
OBSERVE   [step 1 - rate ] : 1 EUR = 1.1383 USD
OBSERVE   [step 2 - calc ] : 250.0 * 1.1383 = 284.575
ANSWER    : 250.0 EUR = 284.575 USD
           (calculation: 250.0 x 1.1383 = 284.575, date: 2026-07-01)
TIME      : 0.55s
```

3.5 Question 5 – MCP and Agent Tools

```
QUESTION : Find information about MCP and explain why it is useful for
           agent tools.
TOOL      : search
OBSERVE   : 5 result(s) returned from DuckDuckGo
ANSWER    : MCP (Model Context Protocol) complements agent orchestration tools
           like LangChain, LangGraph, and CrewAI by serving as a unified
           toolbox from which AI agents can invoke external actions. It enables
           consistent, standardized communication between AI models and tools,
           making it easier to build reliable agentic systems.
TIME      : 2.01s
```

3.6 Question 6 – Compare EUR to GBP and USD

```
QUESTION : Compare EUR to GBP and USD using live exchange-rate data.
TOOL      : multi_currency
OBSERVE   : base=EUR, rates={'GBP': 0.85973, 'USD': 1.1383}
ANSWER    : Live exchange rates for 1 EUR (date: 2026-07-01):
           1 EUR = 0.85973 GBP
           1 EUR = 1.1383 USD
TIME      : 0.57s
```

3.7 Question 7 – Tesla Stock Price

```
QUESTION : What is Tesla's current stock price?
TOOL      : search
OBSERVE   : 5 result(s) returned from DuckDuckGo
ANSWER    : Tesla is set to report Q2 delivery numbers on July 2, with estimates
           ranging from 396,500 to 420,000 vehicles. The company faces
           declining
           U.S. sales, while European markets show strong growth. Analyst
           sentiment
           is mixed. (Source: financial news, 2026-07-02)
TIME      : 1.47s
```

4 Reflection

4.1 What Worked Well

The currency tools worked reliably. Questions 2, 4, and 6 all gave correct, live results using the Frankfurter API. The multi-step logic for Question 4 worked correctly: the agent called the currency tool first, got the rate (1.1383), and then passed it to the calculator. The result (284.575 USD) is correct.

The web search tool (Questions 1, 3, 5, 7) returned useful snippets in every case. DuckDuckGo consistently returned relevant results within about 1–2 seconds.

Tool selection worked automatically for all 7 questions. The keyword-matching approach is simple but effective for the types of questions given.

4.2 What Did Not Work Perfectly

Question 7 (Tesla stock price) returned general financial news rather than an exact current price. This is a known limitation of free web search APIs: they return news snippets, not live stock data. A dedicated stock price API (such as Yahoo Finance or Alpha Vantage) would be needed for a precise answer.

Question 3 asked the agent to summarize in three sentences. The search tool returns snippets as-is, so the formatting into exactly three sentences was not enforced. Adding a post-processing step or a small language model would improve this.

The Frankfurter API v2 endpoint (<https://api.frankfurter.dev/v2/rates>) returned a 422 error during testing. The older stable endpoint (<https://api.frankfurter.app/latest>) was used instead. Both are provided by the same service.

4.3 How the Agent Could Be Improved

A few things that could make this better:

- Add a stock price tool using a dedicated financial API (Yahoo Finance, Alpha Vantage).
- Replace keyword-based tool selection with a small classifier or a prompt to a language model, which would handle more varied phrasing.
- Add a summarization step after web search to produce cleaner, more structured answers.
- Add memory so the agent can refer back to earlier results in a multi-turn conversation.

5 Conclusion

The agentic AI application was implemented successfully in Python. It handles seven different types of questions using three tools: web search, currency conversion, and arithmetic. The agent picks the right tool automatically, shows a full trace of its reasoning, and handles errors gracefully.

The main lesson from this assignment is that even a simple agent with keyword-based routing can be useful and correct for a well-defined set of tasks. The limitations mostly appear at the edges: when the question requires precise real-time data (stock prices) or natural language formatting (three-sentence summaries). Those cases would need more sophisticated components to handle properly.

References

- [1] Z. Xi, W. Chen, X. Guo, et al. *The Rise and Potential of Large Language Model Based Agents: A Survey*. arXiv preprint arXiv:2309.07864, 2023.
- [2] DuckDuckGo. *DDGS Python Library*. https://github.com/deedy5/duckduckgo_search, 2024.
- [3] Frankfurter. *Frankfurter – Exchange Rate API*. <https://www.frankfurter.app>, 2024.